

Mininet and an Intro to SDN & Link and Network with SDN

Project Overview

In part1 and part2 you will learn about Software Defined Networking (SDN). Using Virtualbox, Mininet, and Pox as the implementers of the OpenFlow protocol, you will build simple networks using SDN primitives. In part3 and part4 you will continue learning about Software Defined Networking (SDN), building on your experience from the previous assignment. Using Virtualbox, Mininet, and Pox as the implementers of the OpenFlow protocol, you will build some simple networks using SDN primitives.

- First you will learn to use [mininet](#), a SDN-enabled network emulator.
- For the second portion you will be using POX to implement a simple L2 static firewall via OpenFlow rules determined by your SDN controller.
- For the third portion you will be building an entire network, with multiple switches capable of handling traffic in a static topology.
- In the forth part, you will be modifying your part 3 solution to implement a layer 3 learning IP router that handles ARP and routes traffic dynamically between subnets.

Background

Software Defined Networking and OpenFlow

Software-Defined Networking (SDN) is a recently proposed networking paradigm in which the data and control planes are decoupled from one another. One can think of the control plane as being the network's "brain", i.e., it is responsible for making all decisions, for example, how to forward data, while the data plane is what actually moves the data. In traditional networks, both the control- and data planes are tightly integrated and implemented in the forwarding devices that comprise a network. The SDN control plane is implemented by the "controller" and the data plane by "switches". The controller acts as the "brain" of the network, and sends commands ("rules") to the switches on how to handle traffic. OpenFlow has emerged as the de

facto SDN standard and specifies how the controller and the switches communicate as well as the rules controllers install on switches.

Mininet

Mininet is a software stack that creates a virtual network on your computer/laptop. It accomplishes this task by creating host namespaces (h1, h2, etc) and connecting them through virtual interfaces. So when we run ping between the linux namespaces h1 and h2, the ping will run from h1s namespace through a virtual interface pair created for h1 and h2, before it reaches h2. If h1 and h2 are connected through a switch as shown in the python code in the Mininet walkthrough, the ping will transit multiple virtual interface pairs. The switches that we will be using are running OpenVSwitch (OVS), a software-defined networking stack. Mininet will connect additional virtual interfaces between each virtual port on the switch with each connected host. The host name space allows each host to see the same file system, but operates as its own process that will run separately from each of the other host processes. The OVS version running on the Ubuntu image supports OpenFlow.

POX

Pox is a research/academic implementation of an OpenVFlow controller. It provides python hooks to program mininet-based software emulated networks.

环境配置

本次实验需要重新创立一个虚拟环境，请按照流程重新创立虚拟机。不要用前五次实验的环境!!!

1. 切换到 **mininet-environment** 的目录，运行 **vagrant up** 与 **vagrant ssh** 命令。利用 **ls** 命令查看目录，结果应该如下图所示。

```
vagrant@mnvm:~$ ls
mininet project-1 project-2
```

2. 在虚拟机中运行 **sudo PYTHON=python3 mininet/util/install.sh -a** 来下载 mininet 下载成功后的输出与目录如下：

```
make[1]: Nothing to be done for 'install'.
make[1]: Leaving directory '/home/vagrant/oflops/doc'
Enjoy Mininet!
```

```
vagrant@mnvm:~$ ls
mininet oflops oftest openflow pox project-1 project-2
```

安装成功了可以忽略下面的环境配置部分，直接阅读 **Part1-4**。

3. 如果出现下图的错误:

```
Setting up pkg-config (0.29.1-0ubuntu4) ...
Processing triggers for man-db (2.9.1-1) ...
Cloning into 'openflow'...
fatal: unable to access 'https://github.com/mininet/openflow/': Failed to connect to github.com port 443: Connection timed out
vagrant@mnvm:~$
```

请打开 `install.sh` 文件, 注释掉报错的部分

```
vagrant@mnvm:~$ cd mininet
vagrant@mnvm:~/mininet$ cd util
vagrant@mnvm:~/mininet/util$ vim install.sh

# The following will cause a full OF install, covering:
# -user switch
# The instructions below are an abbreviated version from
# http://www.openflowswitch.org/wk/index.php/Debian_Install
function of {
    echo "Installing OpenFlow reference implementation..."
    cd $BUILD_DIR
    $install autoconf automake libtool make gcc
    if [ "$DIST" = "Fedora" -o "$DIST" = "RedHatEnterpriseServer" ]; then
        $install git pkgconfig glibc-devel
    elif [ "$DIST" = "SUSE LINUX" ]; then
        $install git pkgconfig glibc-devel
    else
        $install git-core autotools-dev pkg-config libc6-dev
    fi
    # was: git clone git://openflowswitch.org/openflow.git
    # Use our own fork on github for now:
    # git clone https://github.com/mininet/openflow
    cd $BUILD_DIR/openflow

    # Patch controller to handle more than 16 switches
    patch -p1 < $MININET_DIR/mininet/util/openflow-patches/controller.patch

    # Resume the install:
    ./boot.sh
```

运行 `git clone https://github.com/mininet/openflow` 手动下载。

下载完成后的目录如下:

```
vagrant@mnvm:~$ ls
mininet openflow project-1 project-2
vagrant@mnvm:~$
```

再次运行更改过的 `install.sh`, `sudo PYTHON=python3 mininet/util/install.sh -a`

然而在实际运行 `install.sh` 时, 可能会出现别的下载错误, 可以仿造上面的例子, 在 `install.sh` 中注释掉报错部分, 手动下载报错的包, 然后再次运行更改后的 `install.sh`。

Assignment

Part 1: Mininet Primer

Learning Objectives (After this section, students will be able to...):

1. Use Mininet, a widely used self-contained network environment for research and experimentation, to build a test network topology.
2. Use the basic debug tools built into Mininet to test connectivity between virtual hosts and get the state of the openflow switches in their topology.

Using Mininet

Using mininet is described [here](#). You can run it by typing: `sudo mn`.

There is also a very helpful [Mininet community wiki](#) with lots of good up-to-date information about how to use mininet effectively.

Inside of the the mininet CLI, try running other commands like help, net, nodes, links and dump. Mininet starts with a default network that you can poke at. Find the mac address of the hosts, the ethernet ports connected, and the hostnames in the system.

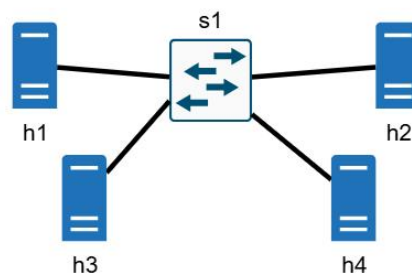
Programming Mininet Topologies

Mininet is also programmable using the python programming language. We have provided some sample topologies [here](#), which should already be downloaded in the VM and unzipped into ~/project-1.

In this directory, you'll find two different directories: topo and pox. Ignore the pox directory for now (it's used in part2). In the topo folder there are a variety of python files. These each define a topology for each of the following project portions. Run the project 1 file with `sudo python3 project-1/topos/part1.py`. It will drop you into the CLI with the network topology defined in the python script.

Task:

Your task in part one is to modify part1.py to represent the following network topology:



After creating the above architecture, your submission should include the following items:

- 1.修改 `part1.py` 来表示以下网络拓扑:
- 2.使用 `sudo python3 project-1/topos/part1.py` 来运行 project 1 文件
- 3.在 CLI 中分别运行 `pingall` , `iperf h1 h4`, `dump`, 这三条命令。然后将输出截图保存, 附在实验报告中。(退出 CLI 可以输入 `exit` 或 `CTRL+Z`)

Part 2: SDN Controllers using POX

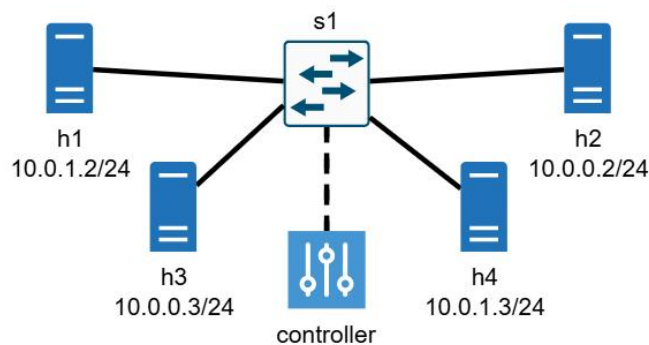
Learning Objectives (After this section, students will be able to...):

- Run their own POX openflow controller to add custom rules to switches in a mininet topology.
- Build an OpenFlow SDN controller program capable of connecting hosts together via a single programmable switch.
- Establish connectivity by flooding packets out switch ports.
- Create match -> action rules to block or allow particular types of traffic through a

switch based on layer 2 and layer 3 header information.

In part 1, we experimented with Mininet using its internal controller. In this (and future) parts, we will instead be using our own controller to send commands to the switches. We will be using the POX controller, which is written in Python.

For this assignment you will create a simple firewall using OpenFlow-enabled switches. The term “firewall” is derived from building construction: a firewall is a wall you place in buildings to stop a fire from spreading. In the case of networking, it is the act of providing security by not letting specified traffic pass through the firewall. This feature is good for minimizing attack vectors and limiting the network “surface” exposed to attackers. In this part, we will provide you with the Mininet topology, part2.py, to setup your network which assumes a remote controller listening on the default IP address and port number 127.0.0.1:6633. **You do not need to (and should not) modify this file.** The topology that this script will setup is as follows. Note that h1 and h4 are on the same subnet and a different one from h2 and h3.



For part 2, we will also provide you with a skeleton POX controller: a1part2controller.py. This file will be where you will make your modifications to create the firewall. The bootstrap should have setup a link from the ~/pox/pox/misc directory to ~/project-1/pox/a1part2controller.py. If no link is present at ~/pox/pox/misc/a1part2controller.py, you can create a new one by running `ln -s ~/project-1/pox/* ~/pox/pox/misc/`. You can then launch the controller with the command `sudo ~/pox/pox.py misc.a1part2controller`. Once the controller is running, you can run mininet with `sudo python3 ~/project-1/topos/part2.py`.

Using a terminal multiplexer like tmux or screen could be handy for running your controller in a separate terminal window within the VM. Alternatively you can just open multiple SSH connections too :)

The rules s1 will need to implement are as follows:

src ip	dst ip	protocol	action
any ipv4	any ipv4	icmp	accept
any	any	arp	accept
any ipv4	any ipv4	-	drop

Basically, your Firewall should allow all ARP and ICMP traffic to pass. However, any

other type of traffic should be dropped. It is acceptable to flood the allowable traffic out all ports. Be careful! Flow tables match the rule with highest priority first, where priority is established based on the order rules are placed in the table. When you create a rule in the POX controller, you need to also have POX “install” the rule in the switch. This makes it so the switch “remembers” what to do for a few seconds. **Do not handle each packet individually inside of the controller! Hint: To do this, look up `ofp_flow_mod`.** The [OpenFlow tutorial](#) (specifically “Sending OpenFlow messages with POX”) and the [POX Wiki](#) are both useful resources for understanding how to use POX.

The Ed board is a great place for swapping resources and asking questions about how to get POX and Mininet working. The goal is to learn networking, not suffer in silence while fighting with a new tool :)

Task:

1. 运行 `ln -s ~/project-1/pox/* ~/pox/pox/misc/`
若报错，可通过 `ls -ld ~/pox/pox/misc/` 查看权限，通过 `sudo chmod 777 ~/pox/pox/misc/` 赋予权限。

```
vagrant@mnvm:~$ ls -ld ~/pox/pox/misc/
drwxr-xr-x 3 root root 4096 Dec  3 10:40 /home/vagrant/pox/pox/misc/

vagrant@mnvm:~$ ln -s ~/project-1/pox/* ~/pox/pox/misc/
ln: failed to create symbolic link '/home/vagrant/pox/pox/misc/a1part2controller.py': Permission denied

vagrant@mnvm:~$ sudo chmod 777 ~/pox/pox/misc/
vagrant@mnvm:~$ ln -s ~/project-1/pox/* ~/pox/pox/misc/
```

2. 修改 `a1part2controller.py` 文件，然后在多个 ssh 连接中，分别使用 `sudo ~/pox/pox.py misc.a1part2controller`，`sudo python3 ~/project-1/topos/part2.py` 来启动控制器和运行 mininet。请注意两条命令的先后顺序。
3. 如果遇到端口号被占用的情况

```
vagrant@mnvm:~$ sudo python3 project-1/topos/part1.py
Traceback (most recent call last):
  File "project-1/topos/part1.py", line 25, in <module>
    net = Mininet(topo=t)
  File "/usr/local/lib/python3.8/dist-packages/mininet/net.py", line 178, in __init__
    self.build()
  File "/usr/local/lib/python3.8/dist-packages/mininet/net.py", line 508, in build
    self.buildFromTopo( self.topo )
  File "/usr/local/lib/python3.8/dist-packages/mininet/net.py", line 475, in buildFromTopo
    self.addController( 'c%d' % i, cls )
  File "/usr/local/lib/python3.8/dist-packages/mininet/net.py", line 291, in addController
    controller_new = controller( name, **params )
  File "/usr/local/lib/python3.8/dist-packages/mininet/node.py", line 1593, in DefaultController
    return controller( name, **kwargs )
  File "/usr/local/lib/python3.8/dist-packages/mininet/node.py", line 1417, in __init__
    self.checkListening()
  File "/usr/local/lib/python3.8/dist-packages/mininet/node.py", line 1433, in checkListening
    raise Exception( "Please shut down the controller which is"
Exception: Please shut down the controller which is running on port 6653:
Active internet connections (servers and established)
```

方法 1: `sudo lsof -i:<port>` 查找占用端口的进程，`sudo kill -9 <pid>` 杀掉进程

方法 2: `sudo mn -c` 重启 mininet

4. 在进入的 CLI 中运行 `pingall`。请注意，h1 和 h4 应该能够互相 ping 通（h2 和 h3 也一样），但不能跨子网。此外，命令 `iperf h1 h4` 应该会失败（因为您阻止了 IP 流量）。这表现为命令挂起。

运行 `dpctl dump-flows`，这应该包含您插入到交换机中的所有规则。请将 2，4 中的输出截图附在实验报告中。

Run the project-2-starter bootstrap-p2.sh script. This script will create links for project2 similar to the ones created in project 1.

cd /home/vagrant/project-2

./bootstrap-p2.sh

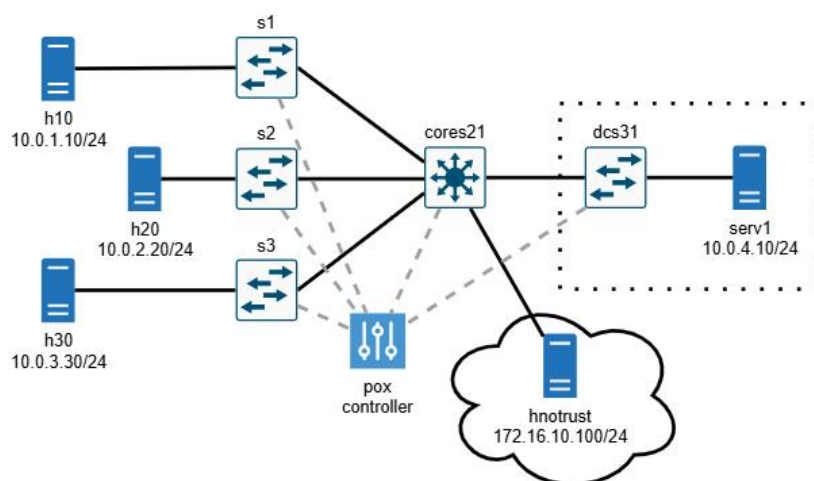
Part 3: A more complex network

Learning Objectives (After this section, students will be able to...):

- Create match -> action rules to forward traffic efficiently out specific ports without flooding according to static knowledge of addresses and topology in the network.
- Create match -> action rules that correctly forward L3 traffic across multiple distinct L2 networks according to a static addressing plan.
- Create an SDN controller program capable of configuring multiple switches with different behavior specified for different types of switches.

In part 2 you implemented a simple firewall that allowed ICMP packets, but blocked all other packets. For this project, you will be expanding on this to implement L3 routing between subnets, and implementing firewalls for certain subnets. The idea is to simulate a (slightly :) more realistic network.

We will be simulating a network for a small company. The company has a 3 floor building, with each floor having its own switch and subnet. Additionally, we have a switch and subnet for all the servers in the data center in the basement, and a core switch connecting everything together. Note that the names and IPs are not to be changed. As with the prior assignment, we have provided the topology (project-2-starter/topos/part1.py) as well as a skeleton controller (project-2-starter/topos/a2part1controller.py). As with part 2 of the previous assignment, you need only modify the controller.



Your goal will be to allow traffic to be transmitted between all the authorized hosts in the company. In this assignment, you will be allowed to flood traffic on the secondary routers (s1,s2,s3,dcs31) in the same method that you did in a1 part2 (using a destination port of of.OFPP_FLOOD). However, in the core router (cores21) you will need to specify specific ports for all IP traffic. You may do this however you choose– however, you may find it easiest

to determine the correct destination port and drop/allow action by using the destination IP address and source IP address, as well as the source port on the switch that the packet originated from.

While all authorized hosts should be able to communicate between themselves and the datacenter server, we need to protect the network from the untrusted Internet. In this scenario we represent the external hosts by hnotrust1, and have the following requirements:

1. We need to block all IP traffic from hnotrust1 to serv1, while still allowing the regular hosts (h10, h20, etc.) to communicate with hnotrust1.
2. To block the Internet from discovering our internal IP addresses, we want to block all ICMP traffic from hnotrust1 to the regular hosts (h10, h20, etc.) and serv1.

In summary of your goals:

- Create a Pox controller (as per a1 part 2) with the following features: All nodes able to communicate **EXCEPT**
 - hnotrust1 cannot send ICMP traffic to h10, h20, h30, or serv1 (but should be able to send IP traffic to hosts).
 - hnotrust1 cannot send any IP traffic to serv1.

Tasks:

1. 修改 `a2part1controller.py` 文件
2. 在多个 ssh 连接中, 分别使用 `sudo ~/pox/pox.py misc.a2part1controller`, `sudo python3 ~/project-2/topos/part1.py` 来启动控制器和运行 mininet
3. 在进入的 CLI 中运行 `pingall`, 除 hnotrust 之外的所有节点都应该能够发送和响应 ping。
4. 运行 `iperf hnotrust1 h10` 和 `iperf h10 serv1`。虽然这些命令中没有显示, 但 hnotrust1 应该无法传输到 serv1, 却能传输带其余节点。
5. 运行 `dpctl dump-flows`。这应该包含您插入到交换机中的所有规则。请将 3, 4, 5 中的输出截图附在实验报告中。

Part 4: A learning router

Learning Objectives (After this section, students will be able to...):

- Write a program to handle ARP requests and generate valid ARP replies to establish connectivity without static address mappings.
- Dynamically create match -> action rules that correctly forward L3 traffic across multiple distinct L2 networks with dynamic L2 addresses.
- Create an SDN controller program capable of holding dynamic state to configure multiple switches in response to traffic observed in the network.
- Reason about the relationship between L2 addresses within a particular subnet and the L3 addresses used for internetwork routing.

For part 2, we'll extend your part 1 code to implement a somewhat more realistic level-3 router out of the cores21 switch. The `a2part2controller.py` skeleton is very similar to part1, and you may want to begin by copying forward some of your functionality from part1. For the topology, we again provide a file (`part2.py`). The difference between `part1.py` and `part2.py` topologies is that there is no longer a static L3<->L2 address mapping loaded into each host, and the default route 'h{N}0-eth0' was changed to 'via 10.0.{N}.1' where '10.0.{N}.1' is the IP address of the gateway (i.e. router) for that particular subnet. This

effectively changes the network from a switched network (with hosts sending to a MAC address) into a routed network (hosts sending to an IP address which may require a gateway out of the L2 network). A minimal a2part1controller will not work on this new topology!

To handle this L2<->L3 mapping cores21 will need to:

- Handle ARP traffic in multiple subnets (without forwarding);
- Generate valid ARP replies when needed; and
- Forward IP traffic *across link domains* (which will require updating the L2 header);

Additionally, this assignment requires that your implementation work dynamically. **You may not install static routes on cores21 at startup.** Instead, your router must learn which ports and which L2 addresses correspond to particular L3 addresses, and install the appropriate rules into the cores21 switch dynamically. This information can be inferred by processing the content of received ARP messages, or possibly other traffic on the network (although processing ARP is sufficient). You may handle each of the individual ARP packets in the controller (i.e., not with flow rules) for part 2, but most IP traffic should be handled with flow rules for efficiency. **The other switches (e.g., s1) do not need to be modified and can continue to flood traffic with static rules.**

For this part of the project, you should drop packets where your router has not learned the appropriate destination. This will mean some packets will have to be dropped that could be routed if your router had more information. See extension 1 to address this shortcoming.

Your implementation should still apply the same L3 policy rules from part 1, particularly that:

1. We need to block all IP traffic from the hnotrust1 subnet to serv1, while still allowing the regular hosts (h10, h20, etc.) to communicate with hnotrust1.
2. To block the Internet from discovering our internal IP addresses, we want to block all ICMP traffic from the hnotrust1 subnet to both the regular hosts (h10, h20, etc.) and serv1.

Note: Combining learning with L3 policies like this is not very secure in practice, since in the real world there is no way to validate the L3 source address with plain IP. Do not naively take the approach used in this assignment and apply it in a production network.

In summary, your complete implementation should:

- Handle ARP traffic in multiple subnets (without forwarding)
- Generate valid ARP replies when needed
- Forward IP traffic *across L2 networks* as well as within them (which may require updating the L2 header)
- Dynamically learn the L3 topology of the network (which subnets are accessible on which ports) via snooping ARP traffic (or another method)
- Allow all hosts to bidirectionally communicate with the server and with each other after the destination addresses have been learned
- Prevent hnotrust1 from sending IP traffic to serv1
- Prevent hnotrust1 from sending ICMP to any of the regular hosts (h10, h20, etc.) or serv1
- Allow bidirectional IP communication between regular hosts (h10, h20, etc.) and hnotrust1

Tasks:

1. 修改 `a2part2controller.py`
2. 在多个 ssh 连接中, 分别使用 `sudo ~/pox/pox.py misc.a2part2controller`, `sudo python3 ~/project-2/topos/part2.py` 来启动控制器和运行 mininet
3. 在进入的 CLI 中运行 `pingall`, 除 `hnotrust` 之外的所有节点都应该能够发送和响应 `ping`。请注意, 当路由器了解路由时, 某些 `ping` 会失败 (为什么会这样?)。
4. 运行 `iperf hnotrust1 h10` 和 `iperf h10 serv1`。`hnotrust` 应该无法转移到 `serv1`, 但应该能够转移到其他主机
5. 运行 `dpctl dump-flows`。这应该包含您插入到交换机中的所有规则。请将 3, 4, 5 中的输出截图附在实验报告中。

注意事项

验收 DDL: 前两个Part 12.30 周二课上; 后两个Part 1.8 周四课上

提交 DDL: 1 月 11日 23: 59 之前发到助教邮箱。

提交材料包括实验报告以及 part1-4 中修改的四个文件。

实验报告中包括 part1-4 的要求截图部分以及关键代码解释。